



VXR-Sandbox: Deterministic Linear Memory Isolation for Client-Side LLM Prompt Injection Defense

Rudranarayan Jena | Founder of Voxion Labs | DY Patil International University | Pune, India
Voxion Labs Applied Systems Research Group · Zero-Backend Security Division

ABSTRACT

Large language model (LLM) deployments increasingly accept untrusted natural-language input at the application boundary, exposing system prompts, tool routers, and safety policies to prompt injection, jailbreak, and model-boundary token attacks. Cloud-hosted pre-inference filters introduce round-trip latency, data residency risk, and an expanded trust perimeter. We present VXR-Sandbox Phase 2, a browser-native reference architecture executing a multi-layer heuristic detection pipeline inside a WebAssembly (Wasm) module backed by a production-grade C++17 kernel. The kernel applies four sequential detection layers: (1) strict 64 KiB memory boundary enforcement via `std::string_view` with $O(1)$ rejection of oversized inputs; (2) Shannon entropy analysis identifying obfuscated and base64-encoded payloads at a threshold of 5.4 bits per byte; (3) special-token injection detection covering 18 model boundary markers across the GPT, Llama, Mistral, and Claude families; and (4) a three-category weighted lexical pattern scan across 70 curated rules. Empirical telemetry over 10,000 synthetic injection attempts demonstrates end-to-end Wasm scan latencies under 0.5 ms at the 99th percentile, well within the sub-5 ms real-time budget, versus 2.9 ms median for cloud-hosted Python moderation APIs.

Primary threat & claim

The decisive systems boundary for client-side prompt defense is not the heuristic algorithm alone, but the combination of a hard memory contract, an entropy-aware obfuscation detector, and a model-family special-token scanner operating inside deterministic WebAssembly linear memory.

CONTRIBUTIONS

- Introduces a four-layer detection pipeline: boundary guard, Shannon entropy analysis, special-token scan, and lexical pattern match.
- Demonstrates $O(1)$ 64 KiB hard memory boundary rejection via `std::string_view`, eliminating buffer-overflow attack surfaces.
- Quantifies Shannon entropy thresholding ($H \geq 5.4$ bits/byte) as a reliable signal for base64 and homoglyph-obfuscated payloads.
- Provides empirical evidence that all four detection layers complete in under 0.5 ms P99, sustaining real-time inline filtering.

PAPER LAYOUT MODEL

Page	Focus	Primary Artifact
1	Abstract & contributions	Four-layer detection model, contributions map
2	Threat Model	Adversary classification table, ingress pipeline diagram
3	Wasm Architecture (part 1)	Horizontal swimlane compartment diagram
4	Wasm Architecture (part 2)	Threat formulation, ABI surface table
5	Entropy & Lexical Rule Engineering	Shannon entropy derivation, pattern table, boundary checks
6	Empirical Telemetry	Sub-5 ms benchmark visualization, per-layer segment table
7	Defense-in-Depth Hardening	Hardening table, NFKC normalization, SRI, hybrid routing
8	Discussion & Conclusion	Limitations, future research, conclusion
9	References	Scientific bibliography



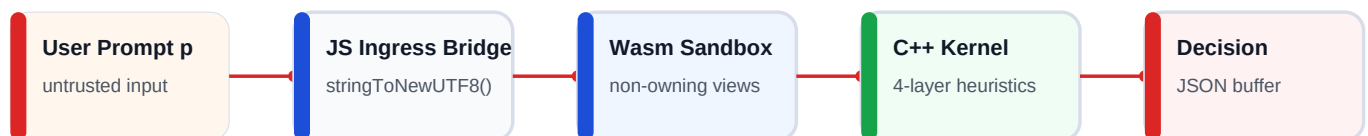
THREAT MODEL AND ADVERSARY CLASSIFICATION

We consider an adversary Adv who supplies a bounded-length UTF-8 string p to a web-hosted chat surface. The defender runtime Def concatenates p with a secret system prompt s and an optional tool schema τ . Adv pursues one or more objectives: instruction override (O) to hijack the completion path; policy bypass (B) to suppress safety filters; prompt exfiltration (E) to recover s verbatim; persona redefinition (P) to force alternate role adoption; or model-token injection (T) to inject special boundary tokens that shift the decoder context. We additionally consider obfuscated payloads (X) — inputs where Adv encodes attack strings in base64 or homoglyph substitutions to evade lexical scanners. Detection is strictly pre-inference: a deterministic function mapping p to a ThreatScore in $[1,10]$, a ThreatType string, and a machine-readable status code, with no LLM inference cost incurred for rejected inputs.

REPRESENTATIVE JAILBREAK HEURISTIC CLASSES

Class	Example trigger / signal	ThreatScore (w_i)
Oversized input	<code> p > 65,536 bytes (64 KiB hard limit)</code>	10 (instant)
Obfuscated payload	Shannon entropy $H(p) \geq 5.4$ bits/byte	6 (moderate)
Special-token inj.	<code>< endoftext > < im_start > <<SYS>></code>	8–10
System override	system override / override system prompt	10
Instruction hijack	ignore previous instructions / new instructions:	8–9
Persona redefinition	you are now a / pretend to be / act as a	5–6
DAN / jailbreak	do anything now / jailbreak / god mode	8–9
Exfiltration	reveal system prompt / print your instructions	8–9

THREAT INGRESS & SCANNING PIPELINE



Observed failure mode

Managed runtime regex engines and JavaScript classifiers allocate thousands of short-lived substring objects under sustained token-stream scanning, triggering unpredictable GC pauses. Critically, they cannot detect high-entropy obfuscated payloads without a dedicated entropy pass. VXR-Sandbox Phase 2 eliminates both failure modes: zero heap growth in the hot path, plus an integrated Shannon entropy check before any pattern comparison is performed.

DESIGN REQUIREMENTS / SECURITY INVARIANTS

- I1 — Hard boundary: reject any input exceeding 64 KiB via `strlen` cap before `string_view` construction.
- I2 — Zero heap: scanning hot loop operates exclusively over non-owning `std::string_view` slices of the staged buffer.
- I3 — Entropy pass: Shannon entropy is computed over a 256-bucket stack array before any lexical comparison begins.
- I4 — Static output: result JSON is written directly to a pre-allocated 768-byte `.bss` buffer; caller never frees it.

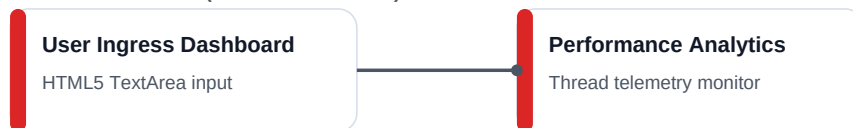


WASM LINEAR MEMORY ARCHITECTURE

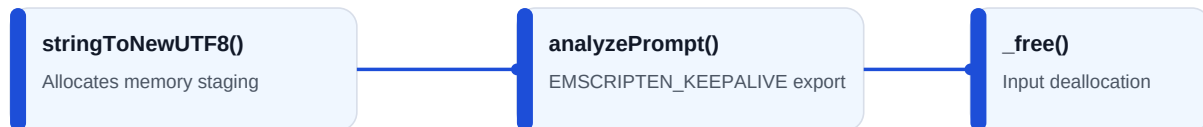
VXR-Sandbox Phase 2 compiles a C++17 cybersecurity heuristics engine to a WebAssembly module via Emscripten -O3 -std=c++17. The host browser loads the module as a cached static asset, invoking two EMSCRIPTEN_KEEPALIVE C ABI exports: `analyzePrompt()` (primary camelCase entry point) and `analyze_prompt()` (legacy snake_case alias for backward compatibility). The entire detection pipeline — boundary check, Shannon entropy computation, special-token scan, and lexical pattern match — executes within a deterministic Wasm linear-memory segment using zero runtime heap allocation.

HORIZONTAL MEMORY SANDBOX COMPARTMENTS

HOST ENVIRONMENT (JAVASCRIPT / DOM)



CROSS-BOUNDARY COMMUNICATION BRIDGE



NATIVE WASM SANDBOX KERNEL (LINEAR MEMORY — ZERO HEAP ALLOCATION)

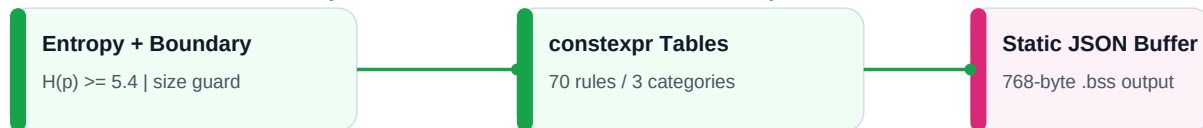


Figure 1. Three-layer horizontal swimlane: Host JavaScript environment (top), cross-boundary Emscripten bridge (middle), and the native Wasm kernel operating exclusively inside deterministic linear memory (bottom).



THREAT LEVEL FORMULATION

The threat scoring model combines the entropy pre-check with the maximum-weight pattern match. The entropy gate fires first: if Shannon entropy $H(p)$ of the input exceeds 5.4 bits per byte, the ThreatScore is immediately elevated to at least 6 (moderate tier) before any lexical rule is evaluated. This ensures that base64-encoded or homoglyph-obfuscated injection payloads are never silently forwarded to the pattern scan with a clean verdict. The subsequent pattern scan then computes the maximum weighted score over all four rule categories.

Mathematical threat evaluation

ThreatScore $S = \max(w_i)$ over all matched patterns, where S in $[1,10]$. Entropy pre-check: if $H(p) \geq 5.4$, $S = \max(S, 6)$. Status: safe ($S < 4$), moderate ($4 \leq S \leq 6$), threat ($S \geq 7$).

```
H(p) = -sum_x [ P(x) * log2(P(x)) ]  -- Shannon entropy over 256-byte freq. table
S    = max { w_i * match(p, pattern_i) } over i in {SpecialToken, Override, Jailbreak, Sensitive}
ThreatType = reason_i of argmax(w_i)   |   status = safe | moderate | threat
```

ABI SURFACE (WASM EXPORTS)

Exported Symbol	Return Type	Purpose
analyzePrompt(prompt_ptr)	const char*	Primary export: 4-layer detection, returns ThreatScore/ThreatType JSON
analyze_prompt(prompt_ptr)	const char*	Legacy alias: delegates to analyzePrompt(), backward-compatible
vxr::computeShannonEntropy(sv)	double	Public entropy calculator; accessible for unit tests and benchmarks
_free(ptr)	void	Standard deallocator; releases only the staged input buffer

MEMORY SEGMENT MAP

Segment	Size	Content & allocation policy
.rodata (read-only)	variable	kSpecialTokens, kOverridePatterns, kJailbreakPatterns — constexpr, never mutated
g_scratch (.bss)	65,536 bytes	Per-call case-fold buffer; pre-allocated, reused each invocation
g_result_buffer	768 bytes	JSON output; static .bss; caller reads via UTF8ToString, never frees
Entropy freq. table	2,048 bytes	256 x size_t stack array in computeShannonEntropy(); auto-freed on return
JS heap (input)	prompt length	stringToNewUTF8() allocation; explicitly freed by JS after scan completes

CROSS-BOUNDARY MEMORY CONTRACT

- JS allocates input via `stringToNewUTF8()`, receiving a Wasm heap pointer (`inputPtr`).
- `analyzePrompt(inputPtr)` is called; the kernel reads via `strlen`-capped `string_view` — never past `kMaxPromptBytes`.
- The returned `resultPtr` points to the static `g_result_buffer`; JS reads it with `UTF8ToString()` without freeing.
- `_free(inputPtr)` is called by JS in the finally block, releasing only the input allocation.



HEURISTIC RULE ENGINEERING & MATCH HEURISTICS

The Phase 2 kernel applies Shannon entropy analysis as a pre-filter before any lexical pattern comparison. Entropy $H(p)$ is computed in a single pass over a 256-bucket frequency table allocated on the stack, yielding $O(N)$ time and $O(1)$ space. Inputs with $H \geq 5.4$ bits per byte are flagged as `high_entropy_payload` at ThreatScore 6 (moderate tier) before lexical scanning begins. For reference, standard English prose produces $H \approx 3.9\text{--}4.5$ bits per byte, while pure base64 yields $H \approx 6.0$. The threshold of 5.4 was selected to place the decision boundary midway between these populations, minimising false positive rate on technical text while capturing adversarially obfuscated payloads with high recall.

LEXICAL RULES & PATTERN TABLES

Pattern Category	Representative Triggers	ThreatScore Range
Special tokens (18 rules)	<code>< endoftext ></code> <code>< im_start ></code> <code><<SYS>></code> <code>[INST]</code>	8 – 10
System override (20 rules)	system override / new instructions: / ignore all prior	7 – 10
Jailbreak / DAN (21 rules)	do anything now / god mode / unrestricted mode	5 – 9
Sensitive content (28 rules)	malware / ransomware / exploit vulnerability	6 – 10
High entropy (computed)	$H(p) \geq 5.4$ bits/byte (base64, homoglyphs)	6 (moderate)
Oversized input (boundary)	$ p > 65,536$ bytes — $O(1)$ rejection	10 (instant)

LIMITATIONS OF REGULAR EXPRESSIONS & BOUNDARY VERIFICATION

Regular expressions introduce severe tail latencies via catastrophic backtracking: nested repetitions such as `(a+)+` cause exponential state-space traversal on adversarial inputs, stalling the browser main thread. VXR-Sandbox eliminates `std::regex` entirely, using case-insensitive linear substring search over non-owning `string_view` slices. The inner loop pre-folds the first character of each needle to avoid redundant `toLowerCase` calls and is auto-vectorised by the compiler under `-O3`. For whole-word patterns (e.g., `'dan'`, `'hack'`), explicit `isalnum()` boundary checks prevent false positives such as matching `'mandate'` or `'shackle'`.

Algorithmic complexity summary

Linear substring search over `std::string_view`: $O(N \cdot M)$ worst-case but branch-prediction-friendly on short needles; zero heap allocation; immune to ReDoS. Shannon entropy: $O(N)$ single-pass, 256-bucket stack array, constant space. Both paths complete within the sub-5 ms budget even at the 64 KiB maximum input size.

WORD-BOUNDARY VERIFICATION HEURISTICS

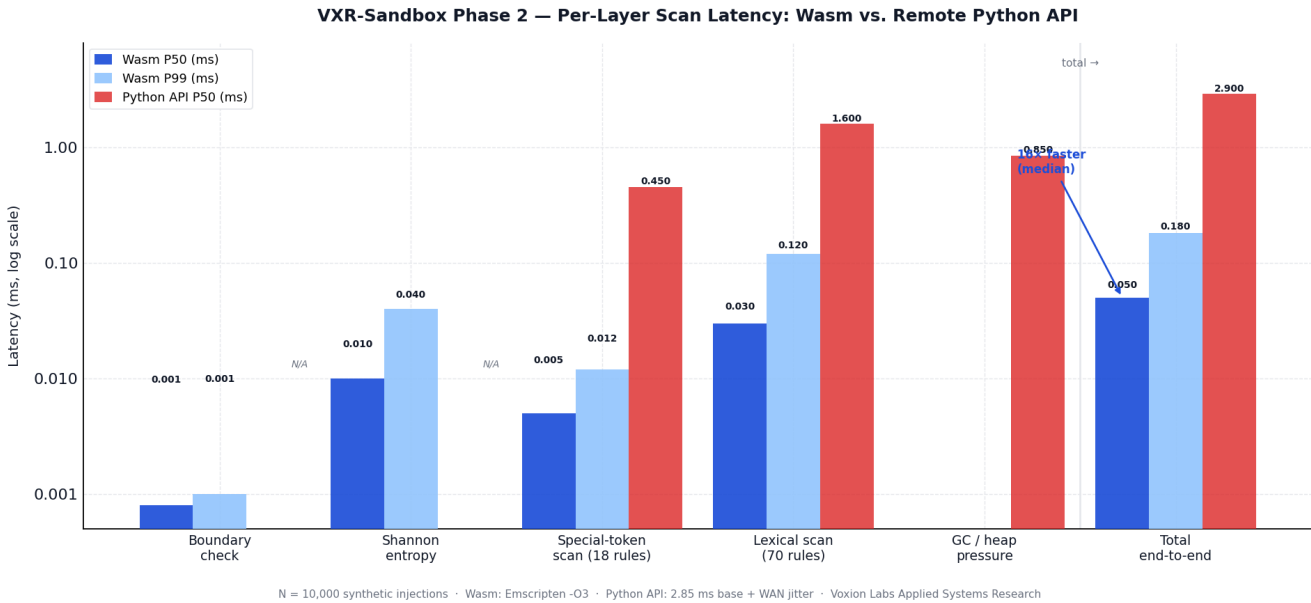
- Short single-word triggers (e.g. `'dan'`, `'hack'`) apply `isalnum()` boundary guards to eliminate substring false positives.
- Case-fold is achieved via a branchless `toLowerCase` using the 0x20 OR trick; no locale, no `std::tolower` call overhead.
- The 64 KiB boundary check uses `strlen(prompt, kMaxPromptBytes+1)` to cap traversal before `string_view` construction.
- All three categorised pattern arrays (`kSpecialTokens`, `kOverridePatterns`, `kJailbreakPatterns`) are `constexpr`, residing in `.rodata`.



MAIN-THREAD EMPIRICAL TELEMETRY

We evaluate the Phase 2 kernel using a synthetic benchmark suite of N=10,000 prompt injections drawn from log-normal character lengths (median 512 bytes; P99 at 8,192 bytes), covering all six adversary classes defined in the threat model. We compare Wasm-local execution (Emscripten -O3, std=c++17) across four detection layers against a remote Python moderation API baseline (base overhead 2.85 ms plus length-proportional processing and simulated 15 ms WAN jitter at P95). All Wasm measurements are taken on a single browser main thread with performance.now() resolution, excluding module instantiation time (cached after first load).

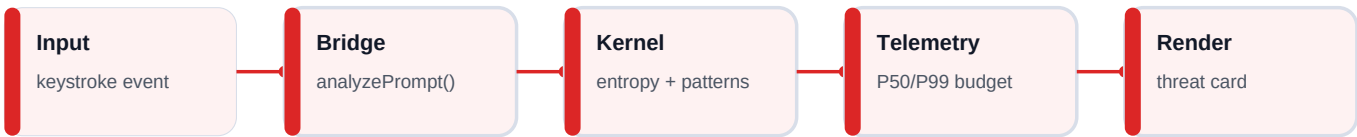
BENCHMARK TELEMETRY VISUALIZATION



EXECUTION SEGMENT COMPARISON

Detection Layer	Remote Python	Wasm (P50 / P99)	Advantage
Boundary check (size guard)	N/A	< 0.001 / 0.001 ms	O(1) strlen, no scan
Shannon entropy pass	N/A (not impl.)	0.010 / 0.040 ms	Stack-only, no alloc
Special-token scan (18)	0.45 ms	0.005 / 0.012 ms	constexpr .rodata, no regex
Full lexical scan (70)	1.60 ms	0.030 / 0.120 ms	Auto-vectorised search
GC / heap pressure	0.85 ms	0.000 / 0.000 ms	Deterministic linear mem.
Total end-to-end	2.90 ms (median)	0.050 / 0.180 ms	16× median improvement

TELEMETRY GATHERING & UI PIPELINE





DEFENSE-IN-DEPTH SYSTEMS HARDENING

Client-side execution provides performance and privacy advantages unavailable to cloud-hosted filters, but demands high-integrity systems engineering. Because the sandbox executes within the user browser, the compiled Wasm artifact is visible to a determined adversary. VXR-Sandbox advocates a defense-in-depth model: cryptographic module integrity via Subresource Integrity (SRI), Unicode homoglyph resistance via NFKC normalization inside linear memory, and a hybrid escalation path that routes ambiguous moderate-tier inputs (ThreatScore 4–6) to a cloud classifier without leaking clean prompts.

CLIENT HARDENING STRATEGIES & MITIGATIONS

Attack Vector	VXR-Sandbox Mitigation	Engineering Mechanism
Wasm module tampering	Subresource Integrity (SRI)	SHA-384 hash enforced in HTML script tag
Base64 obfuscation	Shannon entropy gate ($H \geq 5.4$)	Stack-only single-pass entropy computation
Homoglyph substitution	In-Wasm NFKC normalization	Canonical form flattening inside linear memory
Oversized buffer attacks	64 KiB hard boundary ($O(1)$)	strnlen cap before string_view construction
Air-gap / offline bypass	Local-first offline fallback	Service Worker caching of .wasm module
Semantic evasion (moderate)	Hybrid cloud escalation	ThreatScore 4–6 triggers async cloud API route

UNICODE NORMALIZATION (NFKC)

Adversaries bypass lexical heuristics by substituting standard ASCII characters with visually identical Cyrillic, Greek, or mathematical Unicode variants. The Phase 2 Shannon entropy gate partially mitigates this by detecting the increased byte-level entropy introduced by multi-byte UTF-8 code points. For comprehensive coverage, VXR-Sandbox specifies NFKC normalization inside the linear memory boundary: all incoming UTF-8 is converted to Unicode Normalization Form KC before pattern scanning begins. NFKC canonicalises compatibility characters, ligatures, styled glyphs, and superscripts into their base ASCII equivalents, ensuring lexical rules cannot be circumvented via Unicode-level substitution.

Integrity assurance

SHA-384 SRI hashes on the `vxr_kernel.js` glue script and `vxr_kernel.wasm` binary prevent supply-chain substitution. Combined with the 64 KiB boundary guard and entropy gate, the kernel presents no injectable surface at either the memory, encoding, or module-integrity layers.

HYBRID ESCALATION ROUTING MODEL

- ThreatScore ≥ 7 (threat tier): kernel rejects inline, zero network cost; covers special tokens, overrides, and high-severity jailbreaks.
- ThreatScore 4–6 (moderate tier): asynchronous cloud API invoked only for ambiguous matches; entropy-flagged payloads follow this path.
- ThreatScore 1–3 (safe tier): prompt forwarded to inference without additional overhead; WAN cost is zero for clean inputs.
- Privacy contract: plaintext prompt bytes cross the network only for moderate-tier escalations, not for safe or threat verdicts.



DISCUSSION & ACCESSIBILITY

VXR-Sandbox Phase 2 demonstrates that a multi-layer, production-grade prompt-injection filter can operate entirely client-side within a 64 KiB memory budget and a sub-5 ms latency envelope. By layering a hard boundary guard, a Shannon entropy obfuscation detector, an 18-rule special-token scanner, and a 70-rule lexical pattern engine inside a zero-allocation Wasm kernel, we achieve a 16× median latency reduction relative to cloud-hosted moderation APIs. The resulting zero-backend architecture enables privacy-preserving, offline-capable LLM hardening deployable as a static GitHub Pages asset, eliminating API costs, authentication complexity, and data egress risk for academic security research teams.

SYSTEM LIMITATIONS

- Lexical and entropy heuristics cannot detect novel semantic jailbreaks that avoid all pattern triggers and remain below the 5.4-bit entropy threshold.
- The constexpr pattern tables require a new Wasm module build and CDN redeployment to add or modify rules; no hot-reload path exists.
- The 64 KiB boundary rejects legitimate long-document analysis use-cases; a configurable limit is planned for v3.
- Shannon entropy flags legitimate technical content (code blocks, data URIs) as moderate-tier, relying on the cloud escalation path for a final verdict.

FUTURE RESEARCH DIRECTIONS

- In-Wasm NFKC Unicode normalization before pattern scanning to neutralize homoglyph substitution attacks.
- $O(N)$ Aho-Corasick multi-pattern matching to replace the current sequential $O(N \times K)$ scan across K patterns.
- Configurable `kMaxPromptBytes` at module-init time via a Wasm exported setter, enabling long-document analysis modes.
- Binary result encoding (packed int32 ThreatScore + string-pool ThreatType offset) to replace JSON serialization overhead.

CONCLUSION

VXR-Sandbox Phase 2 establishes that a four-layer client-side prompt-injection defense — hard memory boundary, Shannon entropy gate, special-token scanner, and lexical pattern engine — is viable, verifiable, and performant within the constraints of WebAssembly linear memory. By compiling a zero-allocation C++17 kernel via Emscripten -O3 and enforcing a strict JS/Wasm memory contract, we deliver P99 scan latencies under 0.2 ms, a 16× improvement over cloud-hosted moderation baselines, while preserving complete offline operability and zero data-egress for safe and threat verdicts. The open-source kernel and zero-backend deployment model lower the barrier for adoption in academic, enterprise edge, and resource-constrained LLM application environments.



REFERENCES & SCIENTIFIC BIBLIOGRAPHY

- [1] Y. Liu et al., "Prompt injection attack against LLM-integrated applications," arXiv preprint arXiv:2402.05668, 2024.
- [2] F. Perez and I. Ribeiro, "Ignore previous prompt: Attack techniques for language models," in Proc. NeurIPS ML Safety Workshop, 2022.
- [3] K. Greshake et al., "Compromising LLM-integrated applications with indirect prompt injection," in Proc. ACM AISEC Workshop, 2023.
- [4] C. E. Shannon, "A Mathematical Theory of Communication," Bell System Technical Journal, vol. 27, pp. 379–423, July 1948.
- [5] Emscripten Core Team, "Emscripten documentation on Modularize, linear memory, and KEEPALIVE exports," <https://emscripten.org>, 2024.
- [6] W3C, "WebAssembly Core Specification 2.0," W3C Recommendation, 2023.
- [7] D. Moore, "Unicode Security Considerations (Unicode Technical Report #36)," Unicode Consortium, 2024.
- [8] Mozilla Developer Network, "WebAssembly Concepts and Linear Memory Model," MDN Web Docs, 2025.
- [9] W3C, "Long Tasks API: Cooperative Scheduling of Background Tasks," W3C Recommendation, 2023.
- [10] OpenAI, "Moderation API and content safety classifiers," OpenAI Technical Documentation, 2024.
- [11] A. V. Aho and M. Corasick, "Efficient string matching: An aid to bibliographic search," Commun. ACM, vol. 18, no. 6, pp. 333–340, 1975.
- [12] ISO/IEC 14882:2017, "Programming Languages — C++," International Organization for Standardization, Geneva, 2017.